

Assignment 3: Peptide MS/MS Spectrum Prediction

Release date: 2026-03-01

Due date: 2026-03-15 (before midnight)

1. Objective

Build a model that predicts peptide MS/MS spectra for Orbitrap HCD data. For each peptide and precursor charge, predict singly charged b- and y-ion intensities at each fragmentation site. Ion occurrence is implicitly determined from intensity.

2. Data Source

Training data come from the NIST peptide spectral library (MSP format). The list of the libraries can be found through the following link:

<https://chemdata.nist.gov/dokuwiki/doku.php?id=peptidew:cdownload>

Use Orbitrap HCD libraries. Start with a small dataset such as the Orbitrap mouse (M. musculus) library, then optionally use a larger or different dataset such as the human (H. Sapiens) Orbitrap Fusion HCD synthetic library. Our evaluation will be based on datasets produced on a similar instrument as the ones used for these two libraries.

3. Filtering and Parsing Rules

- Only use fragment ions of type b or y with fragment charge = 1.
- Peptide length must be between 5 and 30 inclusive.
- Cysteine must be CAM (Carbamidomethylation) modified. This rule can be enforced by parsing the first line and see if the CAM modification number matches the number of C in the peptide sequence. E.g.
Name : AAHSQCAYSNPEGTVLLACEESR/3_2 (5,C,CAM) (18,C,CAM)_37.6eV
- Besides Cysteine, all other amino acids must be unmodified.
- Discard spectra that violate these rules.
- Each amino acid is represented by its single-letter code, where C means CAM modified cysteine.

4. Target Representation and Normalization

For a peptide of length L , define fragmentation sites $i = 1..L - 1$. At each site i , the candidate ions are b_i and y_i (charge 1).

Normalization: For each spectrum, collect all charge-1 b and y intensities. Let M be the maximum among them. Normalize each intensity by dividing by M .

Presence threshold: An ion is considered present if normalized intensity $\geq 1e-2$. Any intensity below 0.01 must be written as 0.0 in the output JSON.

For example, in the following screenshots, the left panel shows the intensity values of a spectrum extracted directly from the msp file, and the right panel shows the corresponding normalized values.

```
{
  "sequence": "AAADDGEEPK",
  "precursor_charge": 2,
  "length": 10,
  "sites": [
    [2293.2, 1590.2],
    [54430.0, 15524.5],
    [1470.3, 13081.9],
    [1274.0, 13958.9],
    [0.0, 8465.8],
    [0.0, 0.0],
    [0.0, 8728.8],
    [0.0, 39557.9],
    [0.0, 38281.8]
  ]
}
```

```
{
  "sequence": "AAADDGEEPK",
  "precursor_charge": 2,
  "length": 10,
  "sites": [
    [0.0421, 0.0292],
    [1.0000, 0.2853],
    [0.0270, 0.2404],
    [0.0234, 0.2565],
    [0.0000, 0.1556],
    [0.0000, 0.0000],
    [0.0000, 0.1604],
    [0.0000, 0.7269],
    [0.0000, 0.7033]
  ]
}
```

5. Model Constraints

The model must contain no more than 4,000,000 trainable parameters. Training code must print and save the trainable parameter count.

6. Training/Inference Interface (Required)

Your submission must include two scripts, named a3-train.py (for training) and a3-predict.py (for inference), respectively.

For the training, we will run the following command:

```
python a3-train.py msp_file param_file --epoch num_epoch
```

Your training program will print the number of trainable parameters, and the training performance (binary classification accuracy and MSE of intensity prediction) after each epoch. After num_epoch epochs, it saves parameters to the param_file. We will test this with a small msp_file to test the correctness and reasonableness of your training code.

For the inference (testing), we will run the following command:

```
python a3-predict.py seq_file out_file
```

Input file format (one per line):

seq=PEPTIDE, charge=2

Peptides are guaranteed length 5–30 and contain valid amino acids only. The letter C denotes the CAM modified cysteine.

Your program should load your previously saved weights, make prediction on each sequence, and write the results to a jsonl file.

7. Output Format (JSONL)

The output file must be JSON Lines (JSONL): one JSON object per input line, in the same order.

- sequence: peptide sequence as provided.
- precursor_charge: integer charge parsed from input.
- length: peptide length L.
- sites: list of length L-1; each entry is [b_i_intensity, y_i_intensity].

Before writing JSON, apply the $1e-2$ threshold: intensities below $1e-2$ must be written as 0.0. An example object is shown in the screenshot before (the right panel).

8. Evaluation Metrics

- Binary classification accuracy: presence defined as normalized intensity $\geq 1e-2$.
- RMSE: computed over all candidate ion intensities, including zeros.
- Evaluation will be primarily based on these two metrics. However, programs that take unreasonably long time will be penalized.

9. Required PDF Report

- Description of model architecture and parameter count.
- Data preprocessing details and normalization procedure.
- Performance metrics on held-out data from the training dataset.
- Figures that illustrate the comparison between predicted and experimental spectra for 4 canonical peptides, including peptide, charge state, and full MSP Name line.
- Excluding the figures, we expect 1-2 pages for the rest of the PDF report.

10. Submission Checklist

- a3-predict.py
- a3-train.py
- Saved model weights and configuration
- report.pdf (program description + metrics + 4 peptide comparisons)